



Caching at Database Driver Level

CS631

M.Tech Computer Science and Engineering

By

Dhananjay Andicote (23M0823)

Irshad Khan (23M0824)

Guided By

Prof. S. Sudarshan

Computer Science and Engineering Department

Indian Institute of Technology, Bombay

July 2024

1. Problem Statement and Approach to the Problem

Problem Statement

The project aimed to configure PostgreSQL to act as a local cache for specific relations or parts of relations while ensuring synchronization with a master database. This setup optimizes query performance by serving frequently accessed data locally and dynamically forwarding queries for uncached data to the master database.

Approach to the Problem

- **Publisher / Subscriber:** Master acts as publisher and cache/local db acts as subscriber. Here when an entry is inserted in master db, it is also stored in cache db.
- **Foreign Data Wrappers (FDW):** Set up `postgres_fdw` to enable queries on the master database when requested data is not available locally.
- **Custom Query Interceptor:** Modified the PostgreSQL source code to intercept and reroute `SELECT` and `INSERT` queries dynamically based on caching policies and schema configurations. For `SELECT` it checks if the query result is present in cache, if not then it fetches the query result from master db and inserts the result in cache db. `INSERT` queries ensure data consistency by targeting the master database directly.
- **Partitioning Logic:** A portion of the master database is stored in the local cache database to improve query performance. `SELECT` queries first check the cache for data availability, while uncached queries are routed to the master database.

Logic Flow

The workflow of our project is as follows: when a user submits a query, it is intercepted and checked against the local database. If the result is found in the local database which is synchronized with the master database using a publisher-subscriber model, the query is resolved locally. Otherwise, the query is routed to the master database via a different schema connected through FDW (Foreign Data Wrapper). The obtained result is subsequently appended to the local database schema.

2. Implementation Details

2.1 Logical Replication Setup

Master Database Configuration:

```
1 CREATE PUBLICATION publication_project
2 FOR TABLE project;
```

Local Database Configuration:

```
1 CREATE SUBSCRIPTION subscription_project
2 CONNECTION 'host=localhost port=9432 dbname=postgres
   user=izuku'
3 PUBLICATION publication_project;
```

2.2 FDW Setup

Extension Creation:

```
1 CREATE EXTENSION postgres_fdw;
```

Server Setup:

```
1 CREATE SERVER publisher_server
2 FOREIGN DATA WRAPPER postgres_fdw
3 OPTIONS (host 'localhost', dbname 'postgres', port '9432');
```

User Mapping:

```
1 CREATE USER MAPPING FOR izuku
2 SERVER publisher_server
3 OPTIONS (user 'izuku', password '');
```

Schema Import:

```
1 IMPORT FOREIGN SCHEMA public
2 FROM SERVER publisher_server
3 INTO local_schema;
```

2.3 Custom PostgreSQL Code Modifications

2.3.1 exec_simple_query Function

```
1 static bool exec_simple_query(const char *query_string) {
2     // Success tracking variable
3     bool success = false;
4     int num_rows = 0;
5
6     // ... some other code ...
7
8     // Check if query returned data to forward to master db.
9     num_rows = qc.nprocessed;
10    printf("Number of rows returned: %d\n", num_rows);
11
12    if (num_rows > 0) {
13        success = true;
14    }
15
16    return success;
17 }
```

Listing 2.1: Modified exec_simple_query function

2.3.2 Query Routing in postgres.c

```
1  if (starts_with_keyword(query_string, "INSERT"))
2  {
3      char *modified_query = convert_query(query_string);
4      printf("Modified query Insert: %s\n", modified_query);
5      exec_simple_query(modified_query);
6  }
7  else if (starts_with_keyword(query_string, "SELECT"))
8  {
9      printf("CACHED output:");
10     iscached = exec_simple_query(query_string);
11     printf("Boolean val : %s\n", iscached ? "true" :
12           "false");
13     if(!iscached){
14         char finalQuery[768];
15         char *modified_query =
16             convert_query(query_string);
17         printf("Modified query Select: %s\n",
18               modified_query);
19         // Use FDW query
20         printf("QUERY Not Cached ,caching and returning
21               the result.:\n");
22         transformQuery(modified_query, finalQuery);
23
24         printf("Transformed Query:\n");
25         printf("%s\n", finalQuery);
26
27         exec_simple_query(finalQuery);//retrive from
28               master and cache it locally.
29         exec_simple_query(query_string);//return from
30               cache
31     }
32 }
33 else
34 {
35     exec_simple_query(query_string);
36 }
```

Listing 2.2: Query routing and modifications

2.3.3 Utility Functions

```
1  int starts_with_keyword(const char *query, const char
2      *keyword) {
3      size_t keyword_len = strlen(keyword);
4      while (*query == ' ' || *query == '\t') {
5          query++; // Skip leading whitespace
6      }
```

```

6     return strncasecmp(query, keyword, keyword_len) == 0;
7 }
8
9 char* convert_query(const char *query) {
10     char *new_query;
11     size_t query_len = strlen(query);
12     size_t buffer_size = query_len + 100;
13
14     // Allocate memory for the new query
15     new_query = (char *)malloc(buffer_size);
16     if (new_query == NULL) {
17         perror("Unable to allocate memory for new query");
18         exit(EXIT_FAILURE);
19     }
20
21     const char *current_pos = query;
22     while (*current_pos == ' ' || *current_pos == '\t') {
23         current_pos++; // Skip leading whitespace
24     }
25
26     if (starts_with_keyword(current_pos, "SELECT")) {
27         // Handle SELECT query
28         const char *from_pos = strcasestr(current_pos,
29             "FROM");
30         if (from_pos) {
31             size_t prefix_len = from_pos - query + 5;
32             strncpy(new_query, query, prefix_len);
33             new_query[prefix_len] = '\0';
34
35             strcat(new_query, "local_schema.");
36
37             // Append the rest of the query
38             strcat(new_query, from_pos + 5); // Skip "FROM "
39         } else {
40             strcpy(new_query, query);
41         }
42     } else if (starts_with_keyword(current_pos, "INSERT")) {
43         // Handle INSERT query
44         const char *into_pos = strcasestr(current_pos,
45             "INTO");
46         if (into_pos) {
47             size_t prefix_len = into_pos - query + 5;
48             strncpy(new_query, query, prefix_len);
49             new_query[prefix_len] = '\0';
50             strcat(new_query, "local_schema.");
51             strcat(new_query, into_pos + 5);
52         } else {

```

```

52         strcpy(new_query, query);
53     }
54 } else {
55
56     strcpy(new_query, query);
57 }
58
59
60     return new_query;
61 }
62
63 // Function to transform the query
64 const char *strcasestr_custom(const char *haystack, const
65     char *needle) {
66     if (!haystack || !needle) {
67         return NULL;
68     }
69
70     size_t needleLen = strlen(needle);
71     for (const char *p = haystack; *p; p++) {
72         if (strncasecmp(p, needle, needleLen) == 0) {
73             return p;
74         }
75     }
76     return NULL;
77 }
78
79 // Function to transform the query
80 void transformQuery(const char *inputQuery, char *finalQuery)
81 {
82     // Find the starting position of the table name
83     const char *selectKeyword = "SELECT * FROM ";
84     const char *tableStart = strcasestr_custom(inputQuery,
85         selectKeyword);
86     if (!tableStart) {
87         printf("Invalid query format: Missing 'SELECT *
88             FROM'.\n");
89         return;
90     }
91     tableStart += strlen(selectKeyword);
92
93     // Find the end of table schema and name (up to the first
94         space or semicolon)
95     const char *endOfTable = strchr(tableStart, ' ');
96     if (!endOfTable) {
97         endOfTable = strchr(tableStart, ';'); // Handle if no
98             space but ends with a semicolon
99     }

```

```

94     if (!endOfTable) {
95         printf("Invalid query format: Cannot determine table
           name.\n");
96         return;
97     }
98
99     // Extract the schema and table name
100     size_t tableNameLength = endOfTable - tableStart;
101     char tableName[128] = {0};
102     strncpy(tableName, tableStart, tableNameLength);
103
104     // Create the final INSERT query
105     snprintf(finalQuery, 768, "INSERT INTO public.%s %s",
106             strchr(tableName, '.') ? strchr(tableName, '.')
               + 1 : tableName,
107             inputQuery);
108 }

```

Listing 2.3: Helper functions for query modification

3. Shortcuts Taken and Future Improvements

Shortcuts Taken

- Assumed straightforward query patterns for `SELECT` and `INSERT` queries, leaving edge cases (e.g., complex joins, subqueries, or nested queries) for future implementation.
- Used the `FDW` mechanism to route uncached queries from the local cache database to the master database dynamically, assuming that `postgres_fdw` handles all necessary foreign data access seamlessly.
- Prefixed schema names (`local_schema.`) manually in queries instead of dynamically determining schema names or validating schema existence.
- Relied on simple string operations (`convert_query`, `transform_query`) to match SQL keywords and make necessary query transformation instead of leveraging a full-fledged SQL parser.

Future Improvements

- Integrate advanced caching rules based on usage patterns, query complexity, and frequency of access to optimize cache-hit ratios effectively.
- Extend query routing logic to handle more complex query types such as `UPDATE`, `DELETE`, and multi-table `JOINS`.
- Implement direct database connections using the `libpq` library to fetch results from the master database, bypassing `FDW`, for better control over query execution and error handling.
- Add support for caching partial results of complex queries (e.g., subqueries, aggregate results) and invalidating these cached results dynamically when the underlying data changes.